

Loading required package: ggplot2

Loading required package: lattice

Attaching core tidyverse packages tidyverse 2.0.0

dplyr	1.2.1	readr	2.2.0
forcats	1.0.1	stringr	1.6.0
lubridate	1.9.5	tibble	3.3.1
purrr	1.2.1	tidyr	1.3.2

Conflicts tidyverse_conflicts()

dplyr::filter() masks stats::filter()

dplyr::lag() masks stats::lag()

purrr::lift() masks caret::lift()

Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become

Attaching packages tidymodels 1.5.0

broom	1.0.12	rsample	1.3.2
dials	1.4.3	taylor	0.1.0
infer	1.1.0	tune	2.1.0
modeldata	1.5.1	workflows	1.3.0
parsnip	1.5.0	workflowsets	1.1.1
recipes	1.3.2	yardstick	1.4.0

Conflicts tidymodels_conflicts()

rsample::calibration() masks caret::calibration()

scales::discard() masks purrr::discard()

dplyr::filter() masks stats::filter()

recipes::fixed() masks stringr::fixed()

dplyr::lag() masks stats::lag()

purrr::lift() masks caret::lift()

yardstick::precision() masks caret::precision()

yardstick::recall() masks caret::recall()

yardstick::sensitivity() masks caret::sensitivity()

yardstick::spec() masks readr::spec()

yardstick::specificity() masks caret::specificity()

recipes::step() masks stats::step()

Attaching package: 'janitor'

The following objects are masked from 'package:stats':

```
chisq.test, fisher.test
```

```
Loading required package: Rcpp
```

```
Attaching package: 'Rcpp'
```

```
The following object is masked from 'package:rsample':
```

```
  populate
```

```
This is rstanarm version 2.32.2
```

- See <https://mc-stan.org/rstanarm/articles/priors> for changes to default priors!
- Default priors may change, so it's safest to specify priors, even if equivalent to the default
- For execution on a local, multicore CPU with excess RAM we recommend calling

```
options(mc.cores = parallel::detectCores())
```

```
Attaching package: 'rstanarm'
```

```
The following objects are masked from 'package:caret':
```

```
  compare_models, R2
```

```
Loading required package: Matrix
```

```
Attaching package: 'Matrix'
```

```
The following objects are masked from 'package:tidyr':
```

```
  expand, pack, unpack
```

Loaded glmnet 5.0

Warning message in system2(command = python, args = shQuote(script), stdout = TRUE, :
"running command '"C:/Users/T-GAMER/OneDrive/Documentos/.virtualenvs/r-reticulate/Scripts/python.exe" "C:/Users/T-GAMER/AppData/Local/R/win-library/4.5/reticulate/'

Error in python_config_impl(python) :

Error 103 occurred running C:/Users/T-GAMER/OneDrive/Documentos/.virtualenvs/r-reticulate/Scripts/python.exe:

```
python:      C:/Users/T-GAMER/AppData/Local/R/cache/R/reticulate/uv/cache/archive-
v0/Z7yLGySVAB_TMH-wsihzG/Scripts/python.exe
libpython:   C:/Users/T-GAMER/AppData/Local/R/cache/R/reticulate/uv/python/cpython-
3.12.13-windows-x86_64-none/python312.dll
pythonhome:  C:/Users/T-GAMER/AppData/Local/R/cache/R/reticulate/uv/cache/archive-
v0/Z7yLGySVAB_TMH-wsihzG
virtualenv:   C:/Users/T-GAMER/AppData/Local/R/cache/R/reticulate/uv/cache/archive-
v0/Z7yLGySVAB_TMH-wsihzG/Scripts/activate_this.py
version:     3.12.13 (main, Apr 14 2026, 14:31:26) [MSC v.1944 64 bit (AMD64)]
Architecture: 64bit
numpy:       C:/Users/T-GAMER/AppData/Local/R/cache/R/reticulate/uv/cache/archive-
v0/Z7yLGySVAB_TMH-wsihzG/Lib/site-packages/numpy
numpy_version: 2.5.1
```

NOTE: Python version was forced by py_require()

Warning message in reticulate::py_install(c("pandas", "tabulate", "ipython")):
"An ephemeral virtual environment managed by 'reticulate' is currently in use.
To add more packages to your current session, call `py\_require()` instead
of `py\_install()`. Running:
`py\_require(c("pandas", "tabulate", "ipython"))`"

```
loan <- read_csv("data/Loan/loan_approval_dataset.csv") |>
  clean_names()

#https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset
default <- read_csv("data/default/UCI_Credit_Card.csv")
```

Rows: 4269 Columns: 13
Column specification

Delimiter: ","

chr (3): education, self_employed, loan_status

dbl (10): loan_id, no_of_dependents, income_annum, loan_amount, loan_term, c...

Use `spec()` to retrieve the full column specification for this data.

Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Rows: 30000 Columns: 25

Column specification

Delimiter: ","

dbl (25): ID, LIMIT_BAL, SEX, EDUCATION, MARRIAGE, AGE, PAY_0, PAY_2, PAY_3,...

Use `spec()` to retrieve the full column specification for this data.

Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
loan <- loan %>%
  rename(
    num_dependentes = no_of_dependents,
    escolaridade    = education,
    autonomo        = self_employed,
    renda_anual     = income_annum,
    valor_emprestimo = loan_amount,
    prazo_anos      = loan_term,
    score_credito    = cibil_score,
    ativos_residenciais = residential_assets_value,
    ativos_comerciais  = commercial_assets_value,
    ativos_luxo       = luxury_assets_value,
    ativos_bancarios  = bank_asset_value,
    status            = loan_status
  )

loan <- loan %>%
  dplyr::select(-loan_id)

loan$status <- ifelse(
  loan$status %in% c("Approved", 1, "1"),
  1, 0
)
loan$status <- as.factor(loan$status)
```

```
default <- default %>%
  rename(
    id_cliente = ID,
```

```

    limite_credito = LIMIT_BAL,
    sexo = SEX,
    escolaridade = EDUCATION,
    estado_civil = MARRIAGE,
    idade = AGE,
    atraso_pagto_0 = PAY_0,
    atraso_pagto_2 = PAY_2,
    atraso_pagto_3 = PAY_3,
    atraso_pagto_4 = PAY_4,
    atraso_pagto_5 = PAY_5,
    atraso_pagto_6 = PAY_6,
    fatura_valor_1 = BILL_AMT1,
    fatura_valor_2 = BILL_AMT2,
    fatura_valor_3 = BILL_AMT3,
    fatura_valor_4 = BILL_AMT4,
    fatura_valor_5 = BILL_AMT5,
    fatura_valor_6 = BILL_AMT6,
    pagto_valor_1 = PAY_AMT1,
    pagto_valor_2 = PAY_AMT2,
    pagto_valor_3 = PAY_AMT3,
    pagto_valor_4 = PAY_AMT4,
    pagto_valor_5 = PAY_AMT5,
    pagto_valor_6 = PAY_AMT6,
    PD30 = default.payment.next.month
  )
default <- default %>%
  dplyr::select(-id_cliente)

default$PD30 <- as.factor(default$PD30)

```

```

library(dplyr)
library(readr)
library(tibble)

corr <- default %>%
  select(where(is.numeric)) %>%
  cor(use = "pairwise.complete.obs")

corr_df <- corr %>%
  as.data.frame() %>%
  rownames_to_column("variavel")

```

```
write_csv(corr_df, "correlacao.csv")
```

```
split_loan <- initial_split(
  loan,
  prop   = 0.75,
  strata = status
)

train_data_loan <- training(split_loan)
test_data_loan  <- testing(split_loan)

split_default <- initial_split(
  default,
  prop   = 0.75,
  strata = PD30
)

train_data_default <- training(split_default)
test_data_default  <- testing(split_default)
```

```
log_glm <- logistic_reg() |>
  set_engine("glm") |>
  set_mode("classification")
#lasso
log_l1 <- logistic_reg(
  penalty = tune(),
  mixture = 1
) |>
  set_engine("glmnet") |>
  set_mode("classification")

log_l2 <- logistic_reg(
  penalty = tune(),
  mixture = 0
) |>
  set_engine("glmnet") |>
  set_mode("classification")
```

```
folds <- vfold_cv(train_data_loan, v = 5, strata = status)
grid <- grid_regular(
  penalty(range = c(-6, 1)),
```

```

    levels = 20
  )
metrics <- metric_set(
  accuracy,
  roc_auc,
  f_meas
)

```

```

#
rec_all <- recipe(status ~ ., data = train_data_loan) %>%
  step_dummy(all_nominal_predictors())

rec_def <- recipe(status ~ ., data = train_data_loan) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_numeric_predictors())

rec_score_only <- recipe(
  status ~ score_credito,
  data = train_data_loan
)%>%
  step_dummy(all_nominal_predictors())

rec_score_prazo <- recipe(
  status ~ score_credito + prazo_anos,
  data = train_data_loan
)%>%
  step_dummy(all_nominal_predictors())

rec_no_score <- recipe(
  status ~ .,
  data = train_data_loan
) %>%
  step_dummy(all_nominal_predictors())%>%
  step_rm(score_credito)

rec_no_corr <- recipe(
  status ~ .,
  data = train_data_loan
) %>%
  step_dummy(all_nominal_predictors())%>%
  step_corr(all_numeric_predictors(), threshold = 0.75)

```

```

rec_pca <- recipe(
  status ~ .,
  data = train_data_loan
) %>%
  step_dummy(all_nominal_predictors())%>%
  step_normalize(all_numeric_predictors()) %>%
  step_pca(
    all_numeric_predictors(),
    threshold = 0.95
  )

```

```

wf_logit <- workflow_set(
  preproc = list(
    all_features = rec_all,
    score_only   = rec_score_only,
    #score_prazo = rec_score_prazo,
    no_score     = rec_no_score,
    no_corr      = rec_no_corr
    #pca         = rec_pca
  ),
  models = list(
    logit = log_glm
  )
)

```

```

wf_l1 <- workflow() %>%
  add_recipe(rec_def) %>%
  add_model(log_l1)

```

```

tuned_l1 <- tune_grid(
  wf_l1,
  resamples = folds,
  grid = grid,
  metrics = metrics
)

```

```

wf_l2 <- workflow() %>%
  add_recipe(rec_def) %>%
  add_model(log_l2)

```

```

tuned_l2 <- tune_grid(
  wf_l2,

```

```

    resamples = folds,
    grid = grid,
    metrics = metrics
)

```

→ A | warning: While computing binary `precision()`, no predicted events were detected (i.e. `true\_positive + false\_positive = 0`).
Precision is undefined in this case, and `NA` will be returned.
Note that 242 true event(s) actually occurred for the problematic event level
0

There were issues with some computations A: x5

There were issues with some computations A: x10

There were issues with some computations A: x15

There were issues with some computations A: x20

→ B | warning: While computing binary `precision()`, no predicted events were detected (i.e. `true\_positive + false\_positive = 0`).
Precision is undefined in this case, and `NA` will be returned.
Note that 241 true event(s) actually occurred for the problematic event level
0

There were issues with some computations A: x20

There were issues with some computations A: x20 B: x5

There were issues with some computations A: x20 B: x5

→ A | warning: While computing binary `precision()`, no predicted events were detected (i.e. `true\_positive + false\_positive = 0`).
Precision is undefined in this case, and `NA` will be returned.
Note that 242 true event(s) actually occurred for the problematic event level
0

There were issues with some computations A: x3

There were issues with some computations A: x6

There were issues with some computations A: x9

There were issues with some computations A: x12

→ B | warning: While computing binary `precision()`, no predicted events were detected (i.e. `true\_positive + false\_positive = 0`).
Precision is undefined in this case, and `NA` will be returned.
Note that 241 true event(s) actually occurred for the problematic event level
0

There were issues with some computations A: x12

There were issues with some computations A: x12 B: x3

There were issues with some computations A: x12 B: x3

```
best_l1 <- select_best(tuned_l1, metric = "roc_auc")
final_l1 <- finalize_workflow(wf_l1, best_l1)
fit_l1 <- fit(final_l1, train_data_loan)

best_l2 <- select_best(tuned_l2, metric = "roc_auc")
final_l2 <- finalize_workflow(wf_l2, best_l2)
fit_l2 <- fit(final_l2, train_data_loan)
```

```
library(broom)
```

```
coef_table <- extract_fit_engine(fit_l1) |>
  coef(s = best_l1$penalty)
coef_table
```

```
12 x 1 sparse Matrix of class "dgCMatrix"
               s=0.002069138
(Intercept)      1.600776070
num_dependentes  -0.014641491
renda_anual      -0.730093148
valor_emprestimo  0.770600810
prazo_anos       -0.723077756
```

score_credito	3.840293624
ativos_residenciais	0.003759789
ativos_comerciais	.
ativos_luxo	.
ativos_bancarios	.
escolaridade_Not.Graduate	-0.043215805
autonomo_Yes	0.011867517

```
coef_table <- extract_fit_engine(fit_l2) |>
  coef(s = best_l2$penalty)
coef_table
```

12 x 1 sparse Matrix of class "dgCMatrix"
s=1e-06

(Intercept)	0.866116295
num_dependentes	-0.037874893
renda_anual	-0.096195929
valor_emprestimo	0.151951996
prazo_anos	-0.304510853
score_credito	1.886966643
ativos_residenciais	0.001744485
ativos_comerciais	0.012621253
ativos_luxo	-0.032643489
ativos_bancarios	-0.028225852
escolaridade_Not.Graduate	-0.025509484
autonomo_Yes	0.015458053

```
fits <- wf_logit %>%
  mutate(
    fit = purrr::map(
      info,
      ~ fit(.x$workflow[[1]], data = train_data_loan)
    )
  )
```

```
library(dplyr)
library(purrr)
library(yardstick)
```

```

preds_loan <- fits %>%
  mutate(
    pred_list = map(
      fit,
      ~{
        prob_tbl <- predict(.x, test_data_loan, type = "prob")

        tibble(
          truth = test_data_loan$status,
          class = predict(.x, test_data_loan, type = "class")$.pred_class,
          prob = prob_tbl$.pred_1
        )
      }
    )
  ) %>%
  select(wflow_id, pred_list) %>%
  tidyr::unnest(pred_list)

```

```

library(tibble)
library(yardstick)

#old version

calc_metrics_from_fit <- function(fit, test_data, truth_col = status) {

  # predições
  prob_tbl <- predict(fit, test_data, type = "prob")
  class_tbl <- predict(fit, test_data, type = "class")

  truth <- test_data[[deparse(substitute(truth_col))]]
  class <- class_tbl$.pred_class
  prob <- prob_tbl$.pred_1

  tibble(
    accuracy      = accuracy_vec(truth, class),
    bal_accuracy  = bal_accuracy_vec(truth, class),

    roc_auc       = roc_auc_vec(truth, prob, event_level = "second"),
    log_loss      = mn_log_loss_vec(truth, prob),
    brier         = brier_class_vec(truth, prob),

    sens          = sens_vec(truth, class),

```

```

spec = spec_vec(truth, class),

precision = precision_vec(truth, class),
npv       = npv_vec(truth, class),

f1 = f_meas_vec(truth, class),
f2 = f_meas_vec(truth, class, beta = 2),

kap = kap_vec(truth, class),
mcc = mcc_vec(truth, class),

gmean = sqrt(
  sens_vec(truth, class) * spec_vec(truth, class)
)
}

```

```

calc_metrics_from_fit <- function(fit, test_data, truth_col = status) {

  # Predições
  prob_tbl <- predict(fit, test_data, type = "prob")
  class_tbl <- predict(fit, test_data, type = "class")

  truth <- test_data[[deparse(substitute(truth_col))]]
  class <- class_tbl$.pred_class

  # Classe positiva
  positive_class <- levels(truth)[2]

  # Probabilidade da classe positiva
  prob <- prob_tbl[[paste0(".pred_", positive_class)]]

  # Resposta em 0/1
  truth_num <- as.numeric(truth == positive_class)

  # Evita log(0)
  eps <- 1e-15
  prob_clip <- pmin(pmax(prob, eps), 1 - eps)

  # Log Loss
  log_loss <- -mean(
    truth_num * log(prob_clip) +

```

```

      (1 - truth_num) * log(1 - prob_clip)
    )

    # Brier Score
    brier <- mean((prob - truth_num)^2)

    tibble(
      accuracy      = accuracy_vec(truth, class),
      bal_accuracy  = bal_accuracy_vec(truth, class),

      roc_auc       = roc_auc_vec(truth, prob, event_level = "second"),
      log_loss      = log_loss,
      brier         = brier,

      sens          = sens_vec(truth, class),
      spec          = spec_vec(truth, class),

      precision     = precision_vec(truth, class),
      npv           = npv_vec(truth, class),

      f1            = f_meas_vec(truth, class),
      f2            = f_meas_vec(truth, class, beta = 2),

      kap           = kap_vec(truth, class),
      mcc           = mcc_vec(truth, class),

      gmean         = sqrt(
        sens_vec(truth, class) *
        spec_vec(truth, class)
      )
    )
  }

```

```

metrics_wf <- fits %>%
  mutate(
    metrics = purrr::map(
      fit,
      ~ calc_metrics_from_fit(
        fit = .x,
        test_data = test_data_loan,
        truth_col = status
      )
    )
  )

```

```

    )
  ) %>%
  select(wflow_id, metrics) %>%
  tidyr::unnest(metrics)

metrics_l1 <- calc_metrics_from_fit(
  fit = fit_l1,
  test_data = test_data_loan,
  truth_col = status
) %>%
  mutate(wflow_id = "l1_logit")

metrics_l2 <- calc_metrics_from_fit(
  fit = fit_l2,
  test_data = test_data_loan,
  truth_col = status
) %>%
  mutate(wflow_id = "l2_logit")

metrics_table_final_loan <- bind_rows(
  metrics_wf,
  metrics_l1,
  metrics_l2
)
metrics_table_final_loan

```

A tibble: 6 × 15

	ac- cu-	bal_ac- cu-	roc_auc	log_loss	rier	sens	spec	sion	npv	f1	f2	kap	mcc	gmean
wflow_id	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
all_features_logit	0.92690632	0.92284987	0.62018951	0.87656302	0.75594033	0.97599014	0.73425982	0.37037037	0.50445448	0.80874488	0.79226949			
score_no_score_no_fit	0.92130881	0.91736236	0.62161838	0.87620526	0.73010909	0.97378489	0.72156939	0.33965618	0.50209833	0.81008331	0.81136911	0.72164		
no_score_no_fit	0.62078650	0.62161838	0.92274404	0.72712877	0.59036838	0.16248793	0.68965525	0.73159078	0.37606100	0.3524				
no_fit	0.93670192	0.93736320	0.61840590	0.82058419	0.97599024	0.99048328	0.90909091	0.13129852	0.12085318	0.3277232				
l1_logit	0.92603002	0.92760054	0.99722756	0.82810594	0.93825389	0.92629425	0.13025899	0.45078429	0.68842983	0.219553				
l2_logit	0.92690631	0.92284987	0.62018951	0.87656302	0.75594033	0.97599014	0.73425982	0.37037037	0.50445448	0.80874488	0.79226949			

```
extract_model_summary <- function(wf_fit) {
  model_fit <- workflows::extract_fit_parsnip(wf_fit)

  if (inherits(model_fit$fit, "glm")) {
    return(summary(model_fit$fit))
  }

  if (inherits(model_fit$fit, "lognet")) {
    return(model_fit$fit)
  }

  NULL
}
```

```
model_summaries <- fits %>%
  mutate(
    summary = purrr::map(fit, extract_model_summary)
  ) %>%
  select(wflow_id, summary)

for (i in seq_len(nrow(model_summaries))) {
  cat("\n=====\n")
  cat("Modelo:", model_summaries$wflow_id[i], "\n")
  cat("=====\n\n")
  print(model_summaries$summary[[i]])
}
```

```
=====
Modelo: all_features_logit
=====
```

Call:

```
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.073e+01	4.785e-01	-22.422	< 2e-16 ***
num_dependentes	-2.446e-02	3.988e-02	-0.613	0.540
renda_anual	-5.785e-07	1.028e-07	-5.629	1.82e-08 ***
valor_emprestimo	1.369e-07	2.055e-08	6.662	2.71e-11 ***

prazo_anos	-1.432e-01	1.287e-02	-11.120	< 2e-16 ***
score_credito	2.366e-02	9.076e-04	26.068	< 2e-16 ***
ativos_residenciais	1.262e-08	1.330e-08	0.949	0.343
ativos_comerciais	1.307e-08	1.962e-08	0.666	0.505
ativos_luxo	2.658e-08	1.989e-08	1.336	0.181
ativos_bancarios	3.961e-08	3.804e-08	1.041	0.298
escolaridade_Not.Graduate	-1.679e-01	1.347e-01	-1.247	0.212
autonomo_Yes	6.747e-02	1.338e-01	0.504	0.614

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 4244.0 on 3200 degrees of freedom
 Residual deviance: 1464.3 on 3189 degrees of freedom
 AIC: 1488.3

Number of Fisher Scoring iterations: 7

=====

Modelo: score_only_logit

=====

Call:

stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.092e+01	4.047e-01	-26.98	<2e-16 ***
score_credito	2.070e-02	7.531e-04	27.48	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 4244.0 on 3200 degrees of freedom
 Residual deviance: 1648.5 on 3199 degrees of freedom
 AIC: 1652.5

Number of Fisher Scoring iterations: 6

```
=====
Modelo: no_score_logit
=====
```

```
Call:
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

```
Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)    1.066e+00  1.301e-01   8.195 2.51e-16 ***
num_dependentes -2.676e-02  2.182e-02  -1.227 0.219923
renda_anual     -1.308e-07  5.445e-08  -2.403 0.016276 *
valor_emprestimo 3.854e-08  1.081e-08   3.566 0.000363 ***
prazo_anos      -3.569e-02  6.481e-03  -5.507 3.65e-08 ***
ativos_residenciais -1.693e-09  7.274e-09  -0.233 0.815932
ativos_comerciais  1.164e-08  1.086e-08   1.072 0.283508
ativos_luxo      -5.880e-09  1.092e-08  -0.538 0.590254
ativos_bancarios  7.711e-09  2.159e-08   0.357 0.720953
escolaridade_Not.Graduate -1.839e-02  7.359e-02  -0.250 0.802650
autonomo_Yes     -1.801e-02  7.356e-02  -0.245 0.806565
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
(Dispersion parameter for binomial family taken to be 1)
```

```
Null deviance: 4244.0  on 3200  degrees of freedom
Residual deviance: 4196.2  on 3190  degrees of freedom
AIC: 4218.2
```

```
Number of Fisher Scoring iterations: 4
```

```
=====
Modelo: no_corr_logit
=====
```

```
Call:
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

```
Coefficients:
```

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.046e+01	4.601e-01	-22.724	<2e-16 ***
num_dependentes	-3.690e-02	3.910e-02	-0.944	0.345
prazo_anos	-1.345e-01	1.250e-02	-10.759	<2e-16 ***
score_credito	2.280e-02	8.610e-04	26.484	<2e-16 ***
ativos_residenciais	8.299e-09	1.192e-08	0.696	0.486
ativos_comerciais	3.065e-09	1.793e-08	0.171	0.864
ativos_bancarios	-1.285e-08	2.538e-08	-0.506	0.613
escolaridade_Not.Graduate	-1.510e-01	1.323e-01	-1.141	0.254
autonomo_Yes	9.770e-02	1.314e-01	0.744	0.457

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 4244.0 on 3200 degrees of freedom
 Residual deviance: 1514.8 on 3192 degrees of freedom
 AIC: 1532.8

Number of Fisher Scoring iterations: 7

```
fits_vif <- fits %>%
  dplyr::filter(
    wflow_id %in% c(
      "all_features_logit",
      "no_corr_logit",
      "no_score_logit",
      "feat_select_logit"
    )
  )
```

```
library(car)

calc_vif <- function(wf_fit) {
  wf_fit %>%
    workflows::extract_fit_parsnip() %>%
    pluck("fit") %>%      # objeto glm
    car::vif()
}

vif_results <- fits_vif %>%
  mutate(
```

```
vif = purrr::map(fit, calc_vif)
)
```

Loading required package: carData

Registered S3 method overwritten by 'car':

```
method      from
na.action.merMod lme4
```

Attaching package: 'car'

The following object is masked from 'package:rstanarm':

```
logit
```

The following object is masked from 'package:dplyr':

```
recode
```

The following object is masked from 'package:purrr':

```
some
```

```
vif_tidy <- vif_results %>%
  select(wflow_id, vif) %>%
  tidyr::unnest_longer(vif, values_to = "vif") %>%
  rename(variable = vif_id)
```

```
vif_summary <- vif_tidy %>%
  group_by(wflow_id) %>%
  summarise(
    max_vif = max(vif),
    mean_vif = mean(vif)
  )
vif_summary
```

A tibble: 3 × 3

wflow_id <chr>	max_vif <dbl>	mean_vif <dbl>
all_features_logit	18.667257	4.225453
no_corr_logit	1.586206	1.226418
no_score_logit	17.106545	4.227158

```
library(dplyr)

model_summary <- metrics_table_final_loan %>%
  left_join(
    vif_summary %>%
      group_by(wflow_id) %>%
      summarise(
        max_vif = max(max_vif, na.rm = TRUE),
        mean_vif = mean(mean_vif, na.rm = TRUE)
      ),
    by = "wflow_id"
  )

model_summary
```

A tibble: 6 × 17

[illegible]

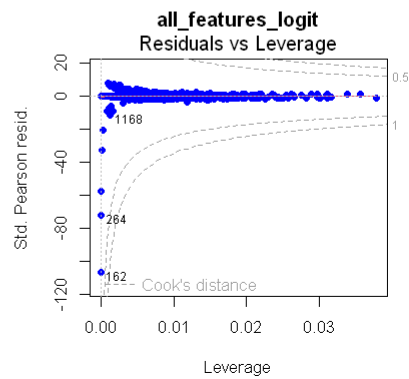
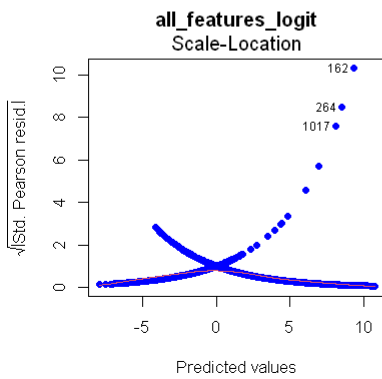
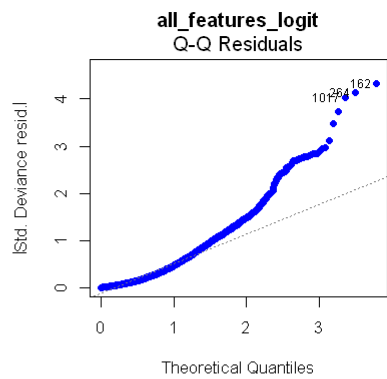
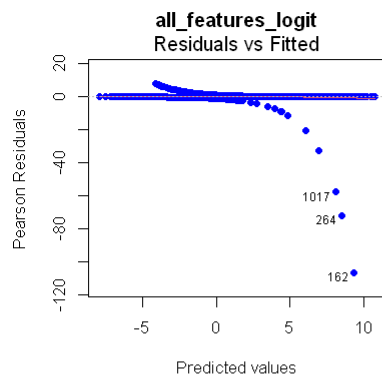
```
get_model <- function(wf_fit) {
  workflows::extract_fit_parsnip(wf_fit)$fit
}
```

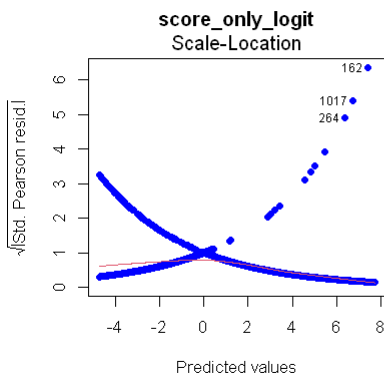
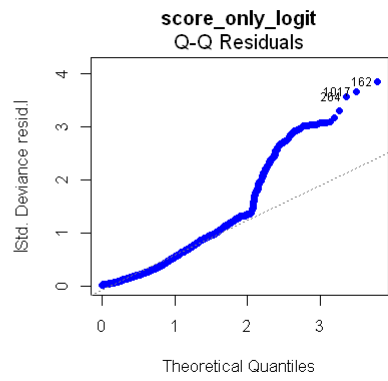
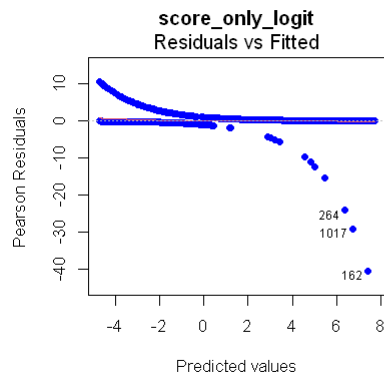
```

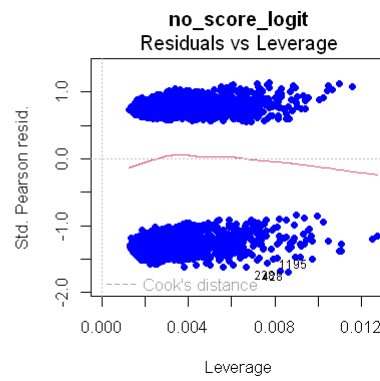
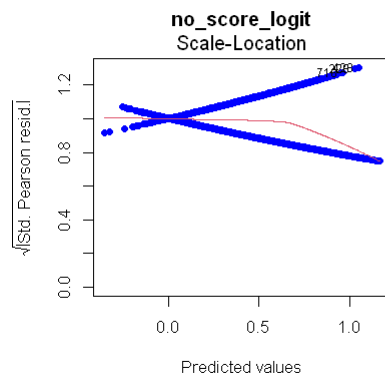
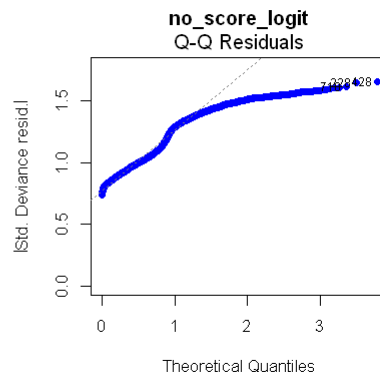
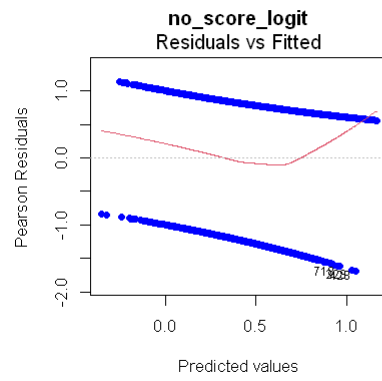
purrr::walk2(
  fits$fit,
  fits$wflow_id,
  ~ {
    modelo <- get_model(.x)

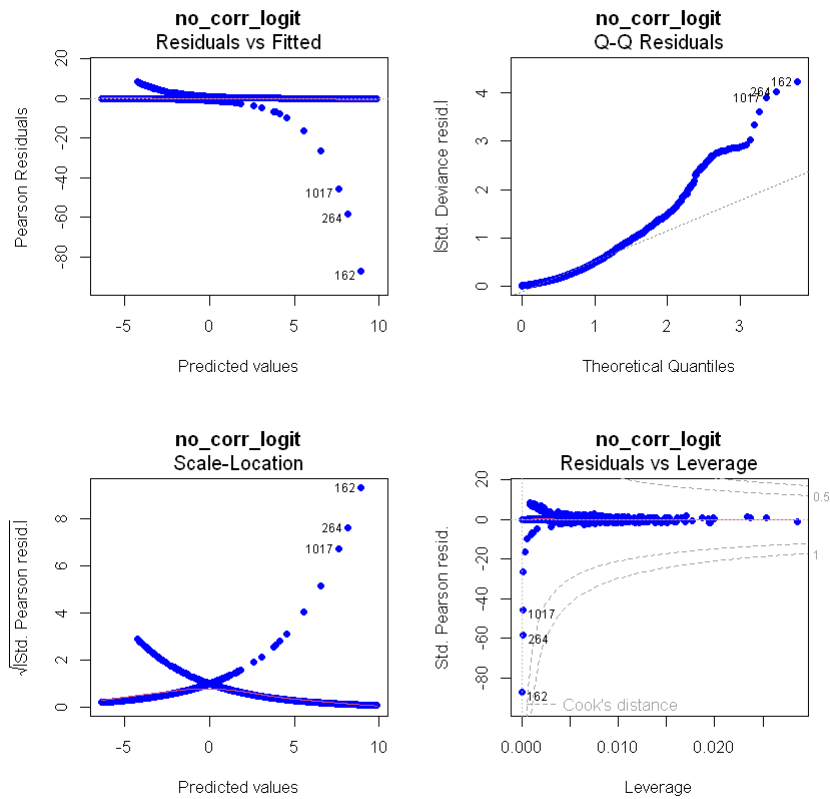
    par(mfrow = c(2, 2))
    plot(
      modelo,
      pch = 19,
      col = "blue",
      main = .y
    )
  }
)

```







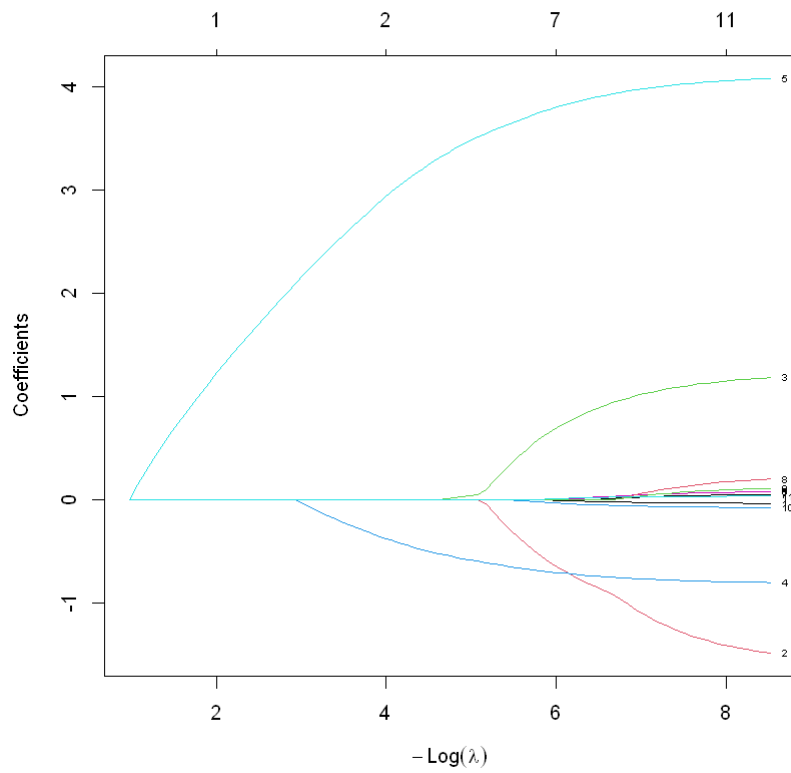


```
glmnet_fit <- fit_l1 %>%
  extract_fit_engine()
plot(glmnet_fit, xvar = "lambda", label = TRUE)

coef_l1 <- coef(glmnet_fit, s = glmnet_fit$lambda.min)

sum(coef_l1 != 0)
```

486

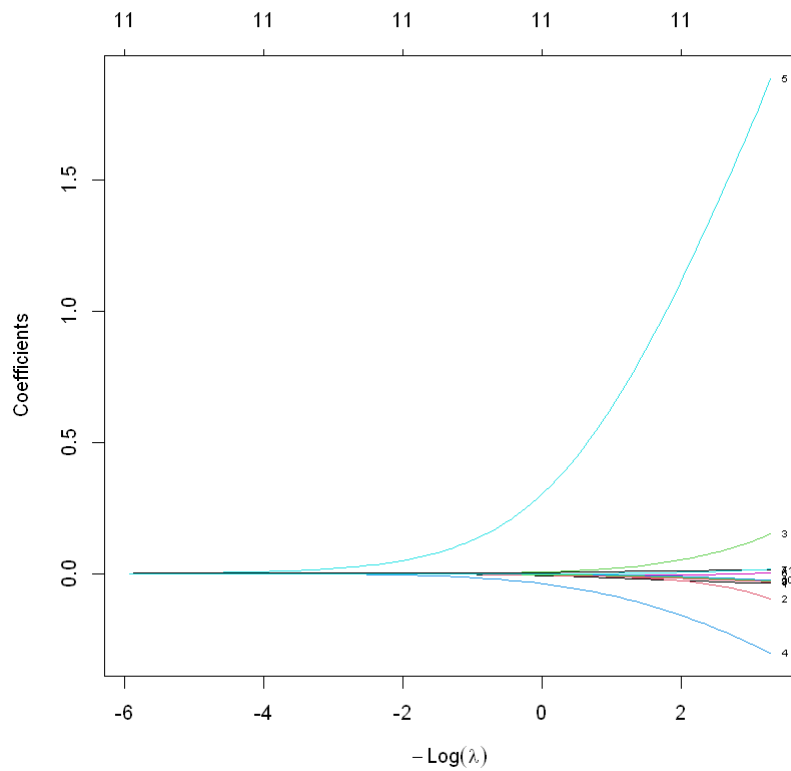


```
glmnet_fit <- fit_12 %>%
  extract_fit_engine()
plot(glmnet_fit, xvar = "lambda", label = TRUE)

coef_12 <- coef(glmnet_fit, s = glmnet_fit$lambda.min)

sum(coef_12 != 0)
```

1200



DEFAULT TIME

```
log_glm <- logistic_reg() |>
  set_engine("glm") |>
  set_mode("classification")
#lasso
log_l1 <- logistic_reg(
  penalty = tune(),
  mixture = 1
) |>
  set_engine("glmnet") |>
  set_mode("classification")

log_l2 <- logistic_reg(
  penalty = tune(),
  mixture = 0
) |>
  set_engine("glmnet") |>
  set_mode("classification")
```

```
folds <- vfold_cv(train_data_default, v = 5, strata = PD30)
```

```
#modelo geral
rec_default <- recipe(PD30 ~ ., data = train_data_default) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_numeric_predictors())
rec_all <- recipe(PD30 ~ ., data = train_data_default) %>%
  step_dummy(all_nominal_predictors())

# Removendo alta correlação
rec_corr <- recipe(
  PD30 ~ .,
  data = train_data_default
) %>% step_dummy(all_nominal_predictors()) %>%
  step_corr(all_numeric_predictors(), threshold = 0.7)

#com pca
rec_pca <- recipe(
  PD30 ~ .,
  data = train_data_default
) %>% step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_numeric_predictors()) %>%
  step_pca(all_numeric_predictors(), threshold = 0.95)
```

```
wf_logit_default <- workflow_set(
  preproc = list(
    all_features = rec_all,
    no_corr      = rec_corr
    #pca         = rec_pca
  ),
  models = list(
    logit = log_glm
  )
)
wf_l1 <- workflow() %>%
  add_recipe(rec_default) %>%
  add_model(log_l1)

tuned_l1 <- tune_grid(
  wf_l1,
  resamples = folds,
```

```

    grid = grid,
    metrics = metrics
  )

wf_l2 <- workflow() %>%
  add_recipe(rec_default) %>%
  add_model(log_l2)

tuned_l2 <- tune_grid(
  wf_l2,
  resamples = folds,
  grid = grid,
  metrics = metrics
)

```

```

best_l1 <- select_best(tuned_l1, metric = "roc_auc")
final_l1 <- finalize_workflow(wf_l1, best_l1)
fit_l1 <- fit(final_l1, train_data_default)

best_l2 <- select_best(tuned_l2, metric = "roc_auc")
final_l2 <- finalize_workflow(wf_l2, best_l2)
fit_l2 <- fit(final_l2, train_data_default)

```

```

library(broom)

coef_table <- extract_fit_engine(fit_l1) |>
  coef(s = best_l1$penalty)
coef_table

```

```

24 x 1 sparse Matrix of class "dgCMatrix"
               s=1e-06
(Intercept)   -1.466639813
limite_credito -0.094661311
sexo          -0.035004786
escolaridade  -0.072950332
estado_civil  -0.089415722
idade         0.055133092
atraso_pagto_0 0.648918405
atraso_pagto_2 0.126711935
atraso_pagto_3 0.095425647
atraso_pagto_4 0.006374048

```

```

atraso_pagto_5  0.063915465
atraso_pagto_6 -0.001422033
fatura_valor_1 -0.328195526
fatura_valor_2  0.089114652
fatura_valor_3  0.072774015
fatura_valor_4  .
fatura_valor_5  0.061297241
fatura_valor_6  .
pagto_valor_1  -0.203277601
pagto_valor_2  -0.169973920
pagto_valor_3  -0.102813317
pagto_valor_4  -0.028684425
pagto_valor_5  -0.031330571
pagto_valor_6  -0.044343025

```

```

coef_table <- extract_fit_engine(fit_l2) |>
  coef(s = best_l2$penalty)
coef_table

```

24 x 1 sparse Matrix of class "dgCMatrix"

```

              s=1e-06
(Intercept)  -1.442426103
limite_credito -0.096639342
sexo         -0.032552769
escolaridade -0.063921471
estado_civil -0.082492064
idade        0.052174196
atraso_pagto_0  0.573329577
atraso_pagto_2  0.144283483
atraso_pagto_3  0.094010299
atraso_pagto_4  0.027315919
atraso_pagto_5  0.056828056
atraso_pagto_6  0.009917793
fatura_valor_1 -0.131804335
fatura_valor_2 -0.023115940
fatura_valor_3  0.003536904
fatura_valor_4 -0.006933742
fatura_valor_5  0.030241967
fatura_valor_6  0.019482572
pagto_valor_1  -0.142934747
pagto_valor_2  -0.122580500
pagto_valor_3  -0.091089024

```

```
pagto_valor_4 -0.037267639
pagto_valor_5 -0.042016896
pagto_valor_6 -0.048021788
```

```
fits <- wf_logit_default %>%
  mutate(
    fit = purrr::map(
      info,
      ~ fit(.x$workflow[[1]], data = train_data_default)
    )
  )
```

```
library(dplyr)
library(purrr)
library(yardstick)

preds <- fits %>%
  mutate(
    preds = purrr::map(
      fit,
      ~{
        prob_tbl <- predict(.x, test_data_default, type = "prob")

        tibble(
          truth = test_data_default$PD30,
          class = predict(.x, test_data_default, type = "class")$.pred_class,
          prob = prob_tbl$.pred_1
        )
      }
    )
  ) %>%
  select(wflow_id, preds)

preds <- preds %>%
  tidyr::unnest(preds)
```

```
metrics_wf <- fits %>%
  mutate(
    metrics = purrr::map(
      fit,
      ~ calc_metrics_from_fit(
        fit = .x,
```

```

      test_data = test_data_default,
      truth_col = PD30
    )
  )
) %>%
select(wflow_id, metrics) %>%
tidyr::unnest(metrics)

metrics_l1 <- calc_metrics_from_fit(
  fit = fit_l1,
  test_data = test_data_default,
  truth_col = PD30
) %>%
  mutate(wflow_id = "l1_logit")

metrics_l2 <- calc_metrics_from_fit(
  fit = fit_l2,
  test_data = test_data_default,
  truth_col = PD30
) %>%
  mutate(wflow_id = "l2_logit")

metrics_table_final <- bind_rows(
  metrics_wf,
  metrics_l1,
  metrics_l2
)
metrics_table_final

```

A tibble: 4 × 15

	ac- cu-	bal_ac- cu-	roc_auc	log_loss	brier	sens	spec	sion	npv	f1	f2	kap	mcc	gmean
wflow_id	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
all_features_logit	0.8076006	0.0523571	0.3650871	0.0594710	0.3968156	0.4230481	0.8102636	0.3586850	0.2339082	0.7373482	0.7373482	0.1324988	0.43536	
no_confusion	0.8074067	0.0510981	0.3671204	0.0592087	0.3967547	0.4230481	0.8106825	0.3586762	0.2337037	0.7373482	0.7373482	0.1324401	0.43108	
l1_logit	0.8074067	0.0428671	0.3235971	0.0314710	0.2968669	0.3990361	0.7708250	0.5886833	0.3418727	0.7179732	0.7179732	0.2382908	0.20657	
l2_logit	0.8038667	0.0935598	0.1136787	0.1807147	0.7579707	0.2163958	0.1348667	0.7358851	0.6834504	0.4247388	0.4247388	0.3062258	0.3233	

```

model_summaries <- fits %>%
  mutate(
    summary = purrr::map(fit, extract_model_summary)
  ) %>%
  select(wflow_id, summary)

for (i in seq_len(nrow(model_summaries))) {
  cat("\n===== \n")
  cat("Modelo:", model_summaries$wflow_id[i], "\n")
  cat("===== \n\n")
  print(model_summaries$summary[[i]])
}

```

```

=====
Modelo: all_features_logit
=====

```

Call:

```
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-6.803e-01	1.378e-01	-4.938	7.91e-07	***
limite_credito	-7.299e-07	1.825e-07	-4.001	6.32e-05	***
sexo	-7.383e-02	3.562e-02	-2.073	0.038188	*
escolaridade	-9.555e-02	2.435e-02	-3.924	8.71e-05	***
estado_civil	-1.739e-01	3.671e-02	-4.738	2.16e-06	***
idade	6.116e-03	2.072e-03	2.952	0.003159	**
atraso_pagto_0	5.762e-01	2.047e-02	28.153	< 2e-16	***
atraso_pagto_2	1.075e-01	2.331e-02	4.609	4.04e-06	***
atraso_pagto_3	7.866e-02	2.615e-02	3.008	0.002629	**
atraso_pagto_4	5.961e-03	2.886e-02	0.207	0.836393	
atraso_pagto_5	6.156e-02	3.115e-02	1.976	0.048116	*
atraso_pagto_6	-8.482e-03	2.586e-02	-0.328	0.742944	
fatura_valor_1	-5.770e-06	1.320e-06	-4.372	1.23e-05	***
fatura_valor_2	2.651e-06	1.744e-06	1.520	0.128504	
fatura_valor_3	1.515e-06	1.503e-06	1.008	0.313512	
fatura_valor_4	-1.054e-06	1.604e-06	-0.657	0.511064	
fatura_valor_5	1.819e-06	1.795e-06	1.013	0.310892	
fatura_valor_6	-1.995e-07	1.346e-06	-0.148	0.882140	

```

pagto_valor_1 -1.422e-05  2.703e-06  -5.260  1.44e-07 ***
pagto_valor_2 -8.207e-06  2.287e-06  -3.588  0.000333 ***
pagto_valor_3 -5.522e-06  2.234e-06  -2.472  0.013420 *
pagto_valor_4 -2.394e-06  1.959e-06  -1.222  0.221734
pagto_valor_5 -1.880e-06  1.933e-06  -0.973  0.330709
pagto_valor_6 -2.641e-06  1.536e-06  -1.719  0.085645 .

```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

```

Null deviance: 23779  on 22499  degrees of freedom
Residual deviance: 20816  on 22476  degrees of freedom
AIC: 20864

```

Number of Fisher Scoring iterations: 5

```

=====
Modelo: no_corr_logit
=====

```

Call:

```
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-6.757e-01	1.375e-01	-4.912	9.00e-07	***
limite_credito	-7.689e-07	1.806e-07	-4.258	2.06e-05	***
sexo	-7.541e-02	3.555e-02	-2.121	0.033901	*
escolaridade	-9.533e-02	2.429e-02	-3.925	8.69e-05	***
estado_civil	-1.742e-01	3.665e-02	-4.753	2.00e-06	***
idade	6.064e-03	2.070e-03	2.930	0.003390	**
atraso_pagto_0	5.857e-01	2.034e-02	28.797	< 2e-16	***
atraso_pagto_2	1.621e-01	1.944e-02	8.336	< 2e-16	***
atraso_pagto_6	7.241e-02	1.757e-02	4.120	3.79e-05	***
fatura_valor_1	-1.730e-06	3.106e-07	-5.569	2.56e-08	***
pagto_valor_1	-9.547e-06	2.277e-06	-4.193	2.76e-05	***
pagto_valor_2	-7.436e-06	2.010e-06	-3.699	0.000217	***
pagto_valor_3	-5.972e-06	1.973e-06	-3.027	0.002470	**
pagto_valor_4	-2.270e-06	1.660e-06	-1.367	0.171663	
pagto_valor_5	-2.221e-06	1.598e-06	-1.390	0.164585	

```
pagto_valor_6 -2.870e-06 1.506e-06 -1.906 0.056622 .
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
(Dispersion parameter for binomial family taken to be 1)
```

```
Null deviance: 23779  on 22499  degrees of freedom  
Residual deviance: 20857  on 22484  degrees of freedom  
AIC: 20889
```

```
Number of Fisher Scoring iterations: 5
```

```
metrics_table_final
```

```
A tibble: 4 × 15
```

	ac- cu-	bal_ac- cu-	roc_auc	log_loss	brier	sens	spec	sion	npv	f1	f2	kap	mcc	gmean
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
all_features_logit	0.8076006	0.05235	0.71365	0.08716	0.05047	0.03968	0.15624	0.23048	0.18102	0.28367	0.35868	0.02339	0.08273	0.13249
no_corr_logit	0.8074067	0.05149	0.71120	0.08720	0.05075	0.03962	0.15679	0.24230	0.18106	0.28825	0.35867	0.02933	0.07734	0.13244
l1_log	0.8074067	0.04286	0.71323	0.05915	0.03147	0.07968	0.69273	0.99038	0.17708	0.85025	0.86833	0.41872	0.71797	0.32382
l2_log	0.8038667	0.03559	0.71136	0.07487	0.07147	0.07757	0.79707	0.42163	0.58134	0.66773	0.88510	0.34504	0.47388	0.30625

```
fits_vif <- fits %>%  
  dplyr::filter(  
    wflow_id %in% c(  
      "all_features_logit",  
      "no_corr_logit"  
    )  
  )  
  
vif_results <- fits_vif %>%  
  mutate(  
    vif = purrr::map(fit, calc_vif)  
  )  
vif_tidy <- vif_results %>%  
  select(wflow_id, vif) %>%  
  tidyr::unnest_longer(vif, values_to = "vif") %>%
```

```

  rename(variable = vif_id)

vif_summary <- vif_tidy %>%
  group_by(wflow_id) %>%
  summarise(
    max_vif = max(vif),
    mean_vif = mean(vif)
  )

```

```

library(dplyr)

model_summary <- metrics_table_final %>%
  left_join(
    vif_summary %>%
      group_by(wflow_id) %>%
      summarise(
        max_vif = max(max_vif, na.rm = TRUE),
        mean_vif = mean(mean_vif, na.rm = TRUE)
      ),
    by = "wflow_id"
  )
model_summary

```

A tibble: 4 × 17

	ac- cu-	bal_ac- cu-	roc_auc	log_loss	sens	spec	sion	npv	f1	f2	kap	mcc	gmean	max_vif	mean_vif
wflow_id	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
all_features_logit	0.8076	0.0052	0.5436	0.0816	0.9971	0.9681	0.0423	0.8681	0.8367	0.8868	0.5033	0.8037	0.3304	98.8643	50.4600
no_confusion_logit	0.8076	0.0052	0.5436	0.0816	0.9971	0.9681	0.0423	0.8681	0.8367	0.8868	0.5033	0.8037	0.3304	98.8643	50.4600
l1_logit	0.8074	0.0042	0.5673	0.3471	0.9934	0.9686	0.2390	0.8677	0.8502	0.8863	0.9941	0.8251	0.9382	98.2063	NA
l2_logit	0.8038	0.0053	0.5598	0.3678	0.9917	0.9675	0.2263	0.8443	0.8677	0.8851	0.9845	0.9473	0.8803	96.2283	NA

```

get_model <- function(wf_fit) {
  workflows::extract_fit_parsnip(wf_fit)$fit
}

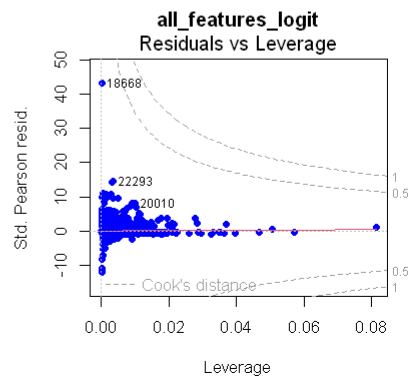
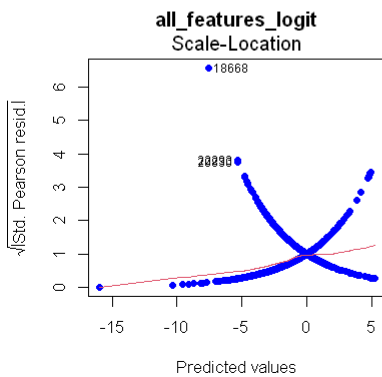
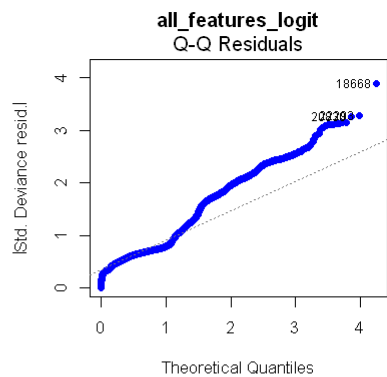
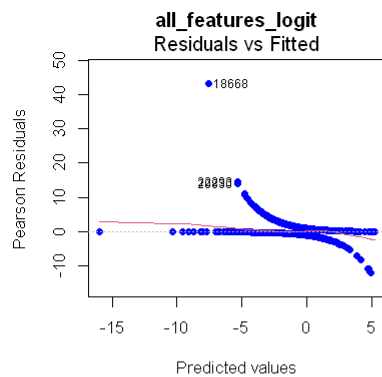
```

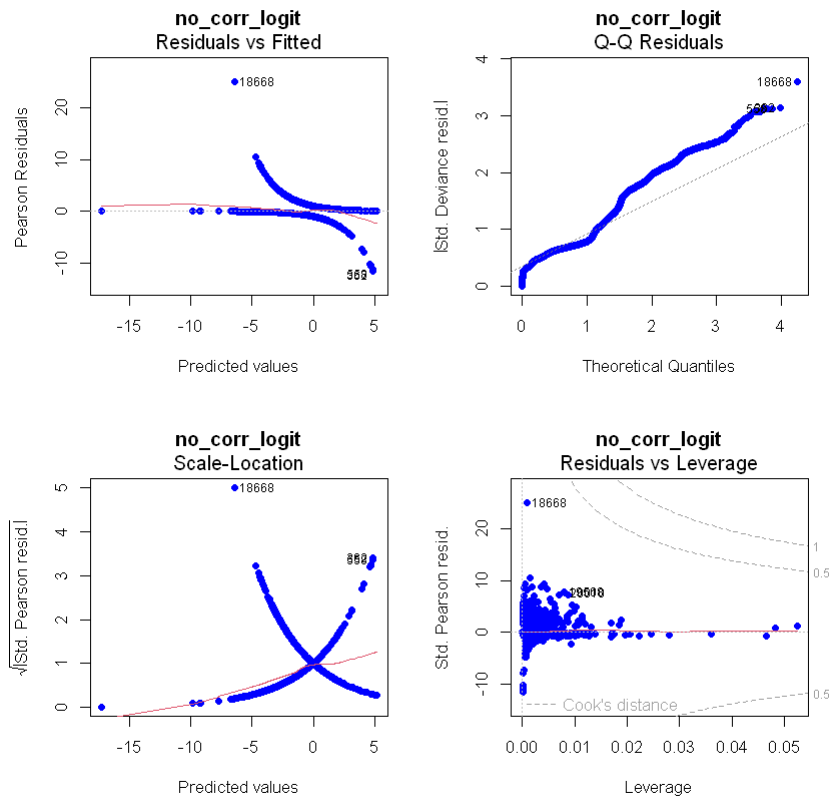
```

purrr::walk2(
  fits$fit,
  fits$wflow_id,
  ~ {
    modelo <- get_model(.x)

    par(mfrow = c(2, 2))
    plot(
      modelo,
      pch = 19,
      col = "blue",
      main = .y
    )
  }
)

```



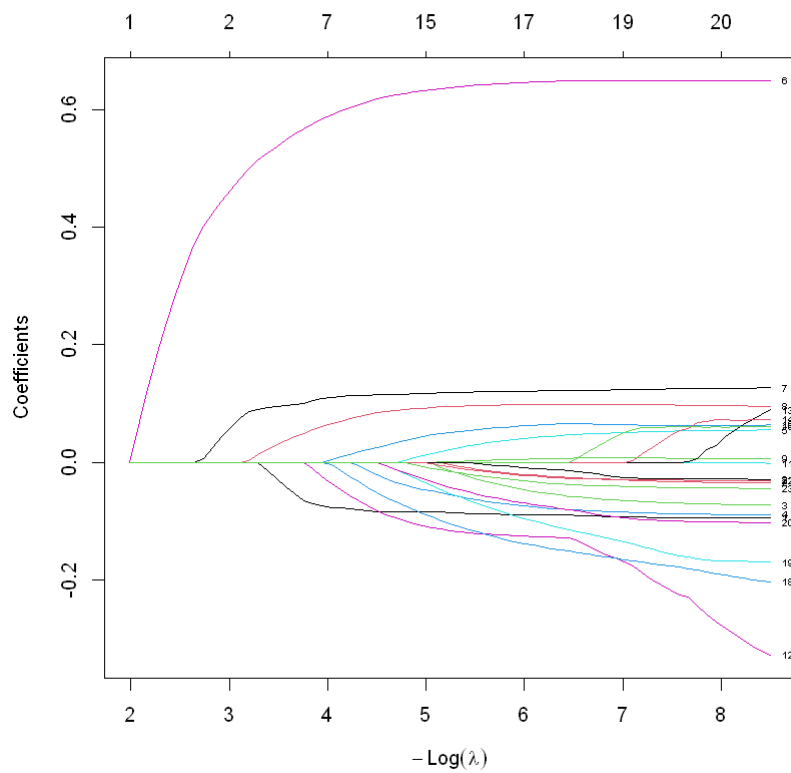


```
glmnet_fit <- fit_l1 %>%
  extract_fit_engine()
plot(glmnet_fit, xvar = "lambda", label = TRUE)

coef_l1 <- coef(glmnet_fit, s = glmnet_fit$lambda.min)

sum(coef_l1 != 0)
```

919

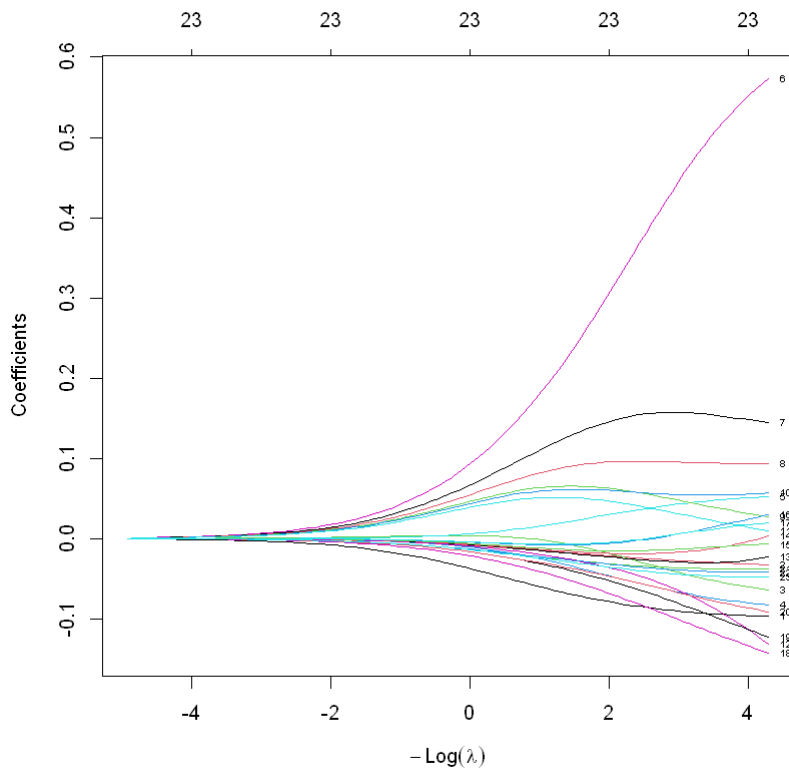


```
glmnet_fit <- fit_l2 %>%
  extract_fit_engine()
plot(glmnet_fit, xvar = "lambda", label = TRUE)

coef_l2 <- coef(glmnet_fit, s = glmnet_fit$lambda.min)

sum(coef_l2 != 0)
```

2400



```
install.packages("Rtools")
```

Installing package into 'C:/Users/T-GAMER/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)

Warning message:

"package 'Rtools' is not available for this version of R"

A version of this package for your version of R might be available elsewhere,
see the ideas at
<https://cran.r-project.org/doc/manuals/r-patched/R-admin.html#Installing-packages>

```
library(rstanarm)
```

```

rec_score <- recipe(status ~ score_credito, data = train_data_loan) %>%
  step_normalize(score_credito)
rec_prep <- prep(rec_score)

train_bayes <- bake(
  rec_prep,
  new_data = NULL
)

test_bayes <- bake(
  rec_prep,
  new_data = test_data_loan
)

```

```

# Não informativa
prior_noninformative <- normal(location = 0, scale = 10, autoscale = FALSE)

fit_p1 <- stan_glm(
  status ~ score_credito,
  data = train_bayes,
  family = binomial(link = "logit"),
  prior = normal(0, 10),
  prior_intercept = normal(0, 10),
  chains = 4,
  iter = 4000,
  seed = 123
)

```

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 0.002757 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 27.57 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 4000 [0%] (Warmup)

Chain 1: Iteration: 400 / 4000 [10%] (Warmup)

Chain 1: Iteration: 800 / 4000 [20%] (Warmup)

Chain 1: Iteration: 1200 / 4000 [30%] (Warmup)

Chain 1: Iteration: 1600 / 4000 [40%] (Warmup)

Chain 1: Iteration: 2000 / 4000 [50%] (Warmup)

```

Chain 1: Iteration: 2001 / 4000 [ 50%] (Sampling)
Chain 1: Iteration: 2400 / 4000 [ 60%] (Sampling)
Chain 1: Iteration: 2800 / 4000 [ 70%] (Sampling)
Chain 1: Iteration: 3200 / 4000 [ 80%] (Sampling)
Chain 1: Iteration: 3600 / 4000 [ 90%] (Sampling)
Chain 1: Iteration: 4000 / 4000 [100%] (Sampling)
Chain 1:
Chain 1: Elapsed Time: 2.186 seconds (Warm-up)
Chain 1:           2.332 seconds (Sampling)
Chain 1:           4.518 seconds (Total)
Chain 1:

```

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 2).

```

Chain 2:
Chain 2: Gradient evaluation took 0.000206 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 2.06 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 4000 [  0%] (Warmup)
Chain 2: Iteration:  400 / 4000 [ 10%] (Warmup)
Chain 2: Iteration:  800 / 4000 [ 20%] (Warmup)
Chain 2: Iteration: 1200 / 4000 [ 30%] (Warmup)
Chain 2: Iteration: 1600 / 4000 [ 40%] (Warmup)
Chain 2: Iteration: 2000 / 4000 [ 50%] (Warmup)
Chain 2: Iteration: 2001 / 4000 [ 50%] (Sampling)
Chain 2: Iteration: 2400 / 4000 [ 60%] (Sampling)
Chain 2: Iteration: 2800 / 4000 [ 70%] (Sampling)
Chain 2: Iteration: 3200 / 4000 [ 80%] (Sampling)
Chain 2: Iteration: 3600 / 4000 [ 90%] (Sampling)
Chain 2: Iteration: 4000 / 4000 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 2.617 seconds (Warm-up)
Chain 2:           2.581 seconds (Sampling)
Chain 2:           5.198 seconds (Total)
Chain 2:

```

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 3).

```

Chain 3:
Chain 3: Gradient evaluation took 0.000156 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 1.56 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:

```

```

Chain 3:
Chain 3: Iteration:    1 / 4000 [  0%] (Warmup)
Chain 3: Iteration:   400 / 4000 [ 10%] (Warmup)
Chain 3: Iteration:   800 / 4000 [ 20%] (Warmup)
Chain 3: Iteration:  1200 / 4000 [ 30%] (Warmup)
Chain 3: Iteration:  1600 / 4000 [ 40%] (Warmup)
Chain 3: Iteration:  2000 / 4000 [ 50%] (Warmup)
Chain 3: Iteration:  2001 / 4000 [ 50%] (Sampling)
Chain 3: Iteration:  2400 / 4000 [ 60%] (Sampling)
Chain 3: Iteration:  2800 / 4000 [ 70%] (Sampling)
Chain 3: Iteration:  3200 / 4000 [ 80%] (Sampling)
Chain 3: Iteration:  3600 / 4000 [ 90%] (Sampling)
Chain 3: Iteration:  4000 / 4000 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 2.267 seconds (Warm-up)
Chain 3:                2.118 seconds (Sampling)
Chain 3:                4.385 seconds (Total)
Chain 3:

```

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 4).

```

Chain 4:
Chain 4: Gradient evaluation took 0.000137 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 1.37 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 4000 [  0%] (Warmup)
Chain 4: Iteration:   400 / 4000 [ 10%] (Warmup)
Chain 4: Iteration:   800 / 4000 [ 20%] (Warmup)
Chain 4: Iteration:  1200 / 4000 [ 30%] (Warmup)
Chain 4: Iteration:  1600 / 4000 [ 40%] (Warmup)
Chain 4: Iteration:  2000 / 4000 [ 50%] (Warmup)
Chain 4: Iteration:  2001 / 4000 [ 50%] (Sampling)
Chain 4: Iteration:  2400 / 4000 [ 60%] (Sampling)
Chain 4: Iteration:  2800 / 4000 [ 70%] (Sampling)
Chain 4: Iteration:  3200 / 4000 [ 80%] (Sampling)
Chain 4: Iteration:  3600 / 4000 [ 90%] (Sampling)
Chain 4: Iteration:  4000 / 4000 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 2.233 seconds (Warm-up)
Chain 4:                2.69 seconds (Sampling)
Chain 4:                4.923 seconds (Total)
Chain 4:

```

```
#Regularizadora fraca
prior_weak <- normal(location = 0, scale = 2.5, autoscale = FALSE)

fit_p2 <- stan_glm(
  status ~ score_credito,
  data = train_bayes,
  family = binomial(link = "logit"),
  prior = normal(0, 2.5),
  prior_intercept = normal(0, 5),
  chains = 4,
  iter = 4000,
  seed = 123
)
```

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 0.000197 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 1.97 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 4000 [0%] (Warmup)

Chain 1: Iteration: 400 / 4000 [10%] (Warmup)

Chain 1: Iteration: 800 / 4000 [20%] (Warmup)

Chain 1: Iteration: 1200 / 4000 [30%] (Warmup)

Chain 1: Iteration: 1600 / 4000 [40%] (Warmup)

Chain 1: Iteration: 2000 / 4000 [50%] (Warmup)

Chain 1: Iteration: 2001 / 4000 [50%] (Sampling)

Chain 1: Iteration: 2400 / 4000 [60%] (Sampling)

Chain 1: Iteration: 2800 / 4000 [70%] (Sampling)

Chain 1: Iteration: 3200 / 4000 [80%] (Sampling)

Chain 1: Iteration: 3600 / 4000 [90%] (Sampling)

Chain 1: Iteration: 4000 / 4000 [100%] (Sampling)

Chain 1:

Chain 1: Elapsed Time: 2.39 seconds (Warm-up)

Chain 1: 2.349 seconds (Sampling)

Chain 1: 4.739 seconds (Total)

Chain 1:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 0.000123 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 1.23 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration: 1 / 4000 [0%] (Warmup)
Chain 2: Iteration: 400 / 4000 [10%] (Warmup)
Chain 2: Iteration: 800 / 4000 [20%] (Warmup)
Chain 2: Iteration: 1200 / 4000 [30%] (Warmup)
Chain 2: Iteration: 1600 / 4000 [40%] (Warmup)
Chain 2: Iteration: 2000 / 4000 [50%] (Warmup)
Chain 2: Iteration: 2001 / 4000 [50%] (Sampling)
Chain 2: Iteration: 2400 / 4000 [60%] (Sampling)
Chain 2: Iteration: 2800 / 4000 [70%] (Sampling)
Chain 2: Iteration: 3200 / 4000 [80%] (Sampling)
Chain 2: Iteration: 3600 / 4000 [90%] (Sampling)
Chain 2: Iteration: 4000 / 4000 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 2.246 seconds (Warm-up)
Chain 2: 2.467 seconds (Sampling)
Chain 2: 4.713 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 3).

Chain 3:
Chain 3: Gradient evaluation took 0.000191 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 1.91 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration: 1 / 4000 [0%] (Warmup)
Chain 3: Iteration: 400 / 4000 [10%] (Warmup)
Chain 3: Iteration: 800 / 4000 [20%] (Warmup)
Chain 3: Iteration: 1200 / 4000 [30%] (Warmup)
Chain 3: Iteration: 1600 / 4000 [40%] (Warmup)
Chain 3: Iteration: 2000 / 4000 [50%] (Warmup)
Chain 3: Iteration: 2001 / 4000 [50%] (Sampling)
Chain 3: Iteration: 2400 / 4000 [60%] (Sampling)
Chain 3: Iteration: 2800 / 4000 [70%] (Sampling)
Chain 3: Iteration: 3200 / 4000 [80%] (Sampling)
Chain 3: Iteration: 3600 / 4000 [90%] (Sampling)
Chain 3: Iteration: 4000 / 4000 [100%] (Sampling)
Chain 3:

```
Chain 3: Elapsed Time: 2.458 seconds (Warm-up)
Chain 3:           3.072 seconds (Sampling)
Chain 3:           5.53 seconds (Total)
Chain 3:
```

```
SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 4).
```

```
Chain 4:
Chain 4: Gradient evaluation took 0.000209 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 2.09 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 4000 [  0%] (Warmup)
Chain 4: Iteration:   400 / 4000 [ 10%] (Warmup)
Chain 4: Iteration:   800 / 4000 [ 20%] (Warmup)
Chain 4: Iteration:  1200 / 4000 [ 30%] (Warmup)
Chain 4: Iteration:  1600 / 4000 [ 40%] (Warmup)
Chain 4: Iteration:  2000 / 4000 [ 50%] (Warmup)
Chain 4: Iteration:  2001 / 4000 [ 50%] (Sampling)
Chain 4: Iteration:  2400 / 4000 [ 60%] (Sampling)
Chain 4: Iteration:  2800 / 4000 [ 70%] (Sampling)
Chain 4: Iteration:  3200 / 4000 [ 80%] (Sampling)
Chain 4: Iteration:  3600 / 4000 [ 90%] (Sampling)
Chain 4: Iteration:  4000 / 4000 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 2.202 seconds (Warm-up)
Chain 4:           2.522 seconds (Sampling)
Chain 4:           4.724 seconds (Total)
Chain 4:
```

```
#Informativa
prior_informative <- normal(location = 0.5, scale = 0.25, autoscale = FALSE)

fit_p3 <- stan_glm(
  status ~ score_credito,
  data = train_bayes,
  family = binomial(link = "logit"),
  prior = normal(0.5, 0.25),
  prior_intercept = normal(0, 2.5),
  chains = 4,
  iter = 4000,
  seed = 123
```

)

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 0.000157 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 1.57 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 4000 [0%] (Warmup)

Chain 1: Iteration: 400 / 4000 [10%] (Warmup)

Chain 1: Iteration: 800 / 4000 [20%] (Warmup)

Chain 1: Iteration: 1200 / 4000 [30%] (Warmup)

Chain 1: Iteration: 1600 / 4000 [40%] (Warmup)

Chain 1: Iteration: 2000 / 4000 [50%] (Warmup)

Chain 1: Iteration: 2001 / 4000 [50%] (Sampling)

Chain 1: Iteration: 2400 / 4000 [60%] (Sampling)

Chain 1: Iteration: 2800 / 4000 [70%] (Sampling)

Chain 1: Iteration: 3200 / 4000 [80%] (Sampling)

Chain 1: Iteration: 3600 / 4000 [90%] (Sampling)

Chain 1: Iteration: 4000 / 4000 [100%] (Sampling)

Chain 1:

Chain 1: Elapsed Time: 2.009 seconds (Warm-up)

Chain 1: 2.012 seconds (Sampling)

Chain 1: 4.021 seconds (Total)

Chain 1:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 0.000133 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 1.33 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 4000 [0%] (Warmup)

Chain 2: Iteration: 400 / 4000 [10%] (Warmup)

Chain 2: Iteration: 800 / 4000 [20%] (Warmup)

Chain 2: Iteration: 1200 / 4000 [30%] (Warmup)

Chain 2: Iteration: 1600 / 4000 [40%] (Warmup)

Chain 2: Iteration: 2000 / 4000 [50%] (Warmup)

Chain 2: Iteration: 2001 / 4000 [50%] (Sampling)

Chain 2: Iteration: 2400 / 4000 [60%] (Sampling)
Chain 2: Iteration: 2800 / 4000 [70%] (Sampling)
Chain 2: Iteration: 3200 / 4000 [80%] (Sampling)
Chain 2: Iteration: 3600 / 4000 [90%] (Sampling)
Chain 2: Iteration: 4000 / 4000 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 1.941 seconds (Warm-up)
Chain 2: 2.026 seconds (Sampling)
Chain 2: 3.967 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 3).

Chain 3:
Chain 3: Gradient evaluation took 0.000165 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 1.65 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration: 1 / 4000 [0%] (Warmup)
Chain 3: Iteration: 400 / 4000 [10%] (Warmup)
Chain 3: Iteration: 800 / 4000 [20%] (Warmup)
Chain 3: Iteration: 1200 / 4000 [30%] (Warmup)
Chain 3: Iteration: 1600 / 4000 [40%] (Warmup)
Chain 3: Iteration: 2000 / 4000 [50%] (Warmup)
Chain 3: Iteration: 2001 / 4000 [50%] (Sampling)
Chain 3: Iteration: 2400 / 4000 [60%] (Sampling)
Chain 3: Iteration: 2800 / 4000 [70%] (Sampling)
Chain 3: Iteration: 3200 / 4000 [80%] (Sampling)
Chain 3: Iteration: 3600 / 4000 [90%] (Sampling)
Chain 3: Iteration: 4000 / 4000 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 1.85 seconds (Warm-up)
Chain 3: 2.276 seconds (Sampling)
Chain 3: 4.126 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 4).

Chain 4:
Chain 4: Gradient evaluation took 0.000139 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 1.39 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:

```

Chain 4: Iteration:    1 / 4000 [  0%] (Warmup)
Chain 4: Iteration:   400 / 4000 [ 10%] (Warmup)
Chain 4: Iteration:   800 / 4000 [ 20%] (Warmup)
Chain 4: Iteration:  1200 / 4000 [ 30%] (Warmup)
Chain 4: Iteration:  1600 / 4000 [ 40%] (Warmup)
Chain 4: Iteration:  2000 / 4000 [ 50%] (Warmup)
Chain 4: Iteration:  2001 / 4000 [ 50%] (Sampling)
Chain 4: Iteration:  2400 / 4000 [ 60%] (Sampling)
Chain 4: Iteration:  2800 / 4000 [ 70%] (Sampling)
Chain 4: Iteration:  3200 / 4000 [ 80%] (Sampling)
Chain 4: Iteration:  3600 / 4000 [ 90%] (Sampling)
Chain 4: Iteration:  4000 / 4000 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 2.081 seconds (Warm-up)
Chain 4:                2.106 seconds (Sampling)
Chain 4:                4.187 seconds (Total)
Chain 4:

```

```
library(yardstick)
```

```

pred_p1 <- posterior_epred(fit_p1, newdata = test_bayes) %>% colMeans()
pred_p2 <- posterior_epred(fit_p2, newdata = test_bayes) %>% colMeans()
pred_p3 <- posterior_epred(fit_p3, newdata = test_bayes) %>% colMeans()

```

```

calc_metrics_bay <- function(truth, class, prob) {

  tibble(
    accuracy      = accuracy_vec(truth, class),
    bal_accuracy  = bal_accuracy_vec(truth, class),

    roc_auc       = roc_auc_vec(truth, prob, event_level = "second"),
    log_loss      = mn_log_loss_vec(truth, prob),

    sens          = sens_vec(truth, class),
    spec          = spec_vec(truth, class),

    precision     = precision_vec(truth, class),
    npv           = npv_vec(truth, class),

    f1            = f_meas_vec(truth, class),
    f2            = f_meas_vec(truth, class, beta = 2),
  )
}

```

```

    kap = kap_vec(truth, class),
    mcc = mcc_vec(truth, class),

    gmean = sqrt(
      sens_vec(truth, class) *
      spec_vec(truth, class)
    )
  )
}

```

```

calc_metrics_bayes <- function(fit, test_data, model_name) {

  # Probabilidade posterior média
  prob <- posterior_epred(
    fit,
    newdata = test_data
  ) %>%
    colMeans()

  # Classificação
  class <- ifelse(
    prob >= 0.5,
    "1",
    "0"
  ) %>%
    factor(
      levels = levels(test_data$status)
    )

  # Verdade
  truth <- test_data$status

  calc_metrics_bay(
    truth = truth,
    class = class,
    prob = prob
  ) %>%
    mutate(
      wflow_id = model_name
    )
}

```

```

metrics_p1 <- calc_metrics_bayes(
  fit = fit_p1,
  test_data = test_bayes,
  model_name = "bayes_prior_1"
)

metrics_p2 <- calc_metrics_bayes(
  fit = fit_p2,
  test_data = test_bayes,
  model_name = "bayes_prior_2"
)

metrics_p3 <- calc_metrics_bayes(
  fit = fit_p3,
  test_data = test_bayes,
  model_name = "bayes_prior_3"
)

```

```

metrics_table_loan_all <- bind_rows(
  metrics_table_final_loan,
  metrics_p1,
  metrics_p2,
  metrics_p3
)
metrics_table_loan_all

```

A tibble: 9 × 15

	ac- cu-	bal_ac- cu-	racy	roc_auc	log_loss	brier	sens	spec	sion	npv	f1	f2	kap	mcc	gmean
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
all_features_logit	0.9269663	0.2284987	0.6201895	0.8765634	0.2759405	0.3975990	0.1477842	0.5082037	0.3705044	0.4814880	0.7448874	0.4887922	0.6949		
score	0.9213088	0.1736256	0.5176236	0.2647273	0.1099013	0.3784892	0.1569393	0.3896551	0.2920983	0.3100833	0.3691172	0.164			
no_score	0.6207865	0.6516621	0.8384619	0.9227444	0.7212897	0.5903483	0.7162480	0.9568905	0.5525573	0.4590783	0.7646100	0.3524			
no_conf	0.9367019	0.2780037	0.3633206	0.8405900	0.2058415	0.3897599	0.0248990	0.4832390	0.9090913	0.2985212	0.8531803	0.277232			
l1_logit	0.9260300	0.2209687	0.6054997	0.2756828	0.1059405	0.3825389	0.2629425	0.1902589	0.4045978	0.4296884	0.2983021	0.19553			
l2_logit	0.9269663	0.1994217	0.4342802	0.6247800	0.6891089	0.4879521	0.3705634	0.7089022	0.5895522	0.4397424	0.4110319	0.4896			
bayes_p1	0.9213088	0.1736256	0.5176236	0.2647273	0.1099013	0.3784892	0.1569393	0.3896551	0.2920983	0.3100833	0.3691172	0.164			
bayes_p2	0.9213088	0.1736256	0.5176236	0.2647273	0.1099013	0.3784892	0.1569393	0.3896551	0.2920983	0.3100833	0.3691172	0.164			

	ac- cu-	bal_ac- cu-						pre- ci-									
wflow_ric	racy	roc_auc	log_loss	brier	sens	spec	sion	npv	f1	f2	kap	mcc	gmean				
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
bayes_0	0.9194737	0.7148879	0.6512676	0.31411	NA	0.8960896	0.837948	0.916256	0.365589	0.382789	0.515382	0.897182	0.897761	0.446931			

```
posterior_interval(fit_p1)
posterior_interval(fit_p3)
```

A matrix: 2 × 2 of type dbl

	5%	95%
(Intercept)	1.380859	1.649898
score_credito	3.394910	3.829870

A matrix: 2 × 2 of type dbl

	5%	95%
(Intercept)	1.176178	1.399741
score_credito	2.882466	3.204822

```
prob_p1 <- posterior_epred(
  fit_p1,
  newdata = test_data_loan
) %>%
  colMeans()

pred_p1 <- tibble(
  truth = test_data_loan$status,
  class = ifelse(prob_p1 >= 0.5, "1", "0"),
  prob = prob_p1
)
```

```
summary(fit_p1)
```

Model Info:

```

function:      stan_glm
family:        binomial [logit]
formula:       status ~ score_credito
algorithm:     sampling
sample:        8000 (posterior sample size)
priors:        see help('prior_summary')
observations:  3201
predictors:    2

```

Estimates:

	mean	sd	10%	50%	90%
(Intercept)	1.5	0.1	1.4	1.5	1.6
score_credito	3.6	0.1	3.4	3.6	3.8

Fit Diagnostics:

	mean	sd	10%	50%	90%
mean_PPD	0.6	0.0	0.6	0.6	0.6

The mean_ppd is the sample average posterior predictive distribution of the outcome variable

MCMC diagnostics

	mcse	Rhat	n_eff
(Intercept)	0.0	1.0	2527
score_credito	0.0	1.0	2706
mean_PPD	0.0	1.0	5793
log-posterior	0.0	1.0	2560

For each parameter, mcse is Monte Carlo standard error, n_eff is a crude measure of effective

```

library(recipes)

rec_default <- recipe(PD30 ~ ., data = train_data_default) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_numeric_predictors())
rec_prep <- prep(rec_default)

```

```

train_bayes <- bake(
  rec_prep,
  new_data = NULL
)

```

```

test_bayes <- bake(

```

```

rec_prep,
new_data = test_data_default
)

```

```

fit_or_load <- function(path, fit_expr) {

  if (file.exists(path)) {

    message("Modelo carregado de: ", path)
    return(readRDS(path))

  }

  message("Treinando modelo...")

  fit <- eval.parent(substitute(fit_expr))

  saveRDS(fit, path)

  message("Modelo salvo em: ", path)

  fit
}

```

```

dir.create("models", showWarnings = FALSE)

```

```

fit_normal <- fit_or_load(
  "models/fit_prior_normal.rds",
  stan_glm(
    PD30 ~ .,
    data = train_bayes,
    family = binomial(link = "logit"),

    prior = normal(
      location = 0,
      scale = 1,
      autoscale = FALSE
    ),

    prior_intercept = normal(
      location = 0,
      scale = 5,

```

```

      autoscale = FALSE
    ),

    chains = 4,
    iter = 1000,
    warmup = 500,
    seed = 123
  )
)

```

Modelo carregado de: models/fit_prior_normal.rds

```

fit_laplace <- fit_or_load(
  "models/fit_prior_laplace.rds",
  stan_glm(
    PD30 ~ .,
    data = train_bayes,
    family = binomial(link = "logit"),

    prior = laplace(
      location = 0,
      scale = 1,
      autoscale = FALSE
    ),

    prior_intercept = normal(
      0,
      5,
      autoscale = FALSE
    ),

    chains = 4,
    iter = 1000,
    warmup = 500,
    seed = 123
  )
)

```

Modelo carregado de: models/fit_prior_laplace.rds

```

fit_horseshoe <- fit_or_load(
  "models/fit_prior_horseshoe.rds",
  stan_glm(
    PD30 ~ .,
    data = train_bayes,
    family = binomial(link = "logit"),

    prior = hs(),

    prior_intercept = normal(
      0,
      5,
      autoscale = FALSE
    ),

    chains = 4,
    iter = 1000,
    warmup = 500,
    seed = 123
  )
)

```

Modelo carregado de: models/fit_prior_horseshoe.rds

```

pred_hs <- posterior_epred(fit_horseshoe, newdata = test_bayes) %>% colMeans()
pred_lp <- posterior_epred(fit_laplace, newdata = test_bayes) %>% colMeans()
pred_norm <- posterior_epred(fit_normal, newdata = test_bayes) %>% colMeans()

```

```

calc_metrics_bayes <- function(fit, test_data, model_name) {

  # Probabilidade posterior média
  prob <- posterior_epred(
    fit,
    newdata = test_data
  ) %>%
    colMeans()

  # Classificação
  class <- ifelse(
    prob >= 0.5,
    "1",

```

```

    "0"
  ) %>%
    factor(
      levels = levels(test_data$PD30)
    )

# Verdade
truth <- test_data$PD30

calc_metrics_bay(
  truth = truth,
  class = class,
  prob = prob
) %>%
  mutate(
    wflow_id = model_name
  )
}

```

```

metrics_p1 <- calc_metrics_bayes(
  fit = fit_horseshoe,
  test_data = test_bayes,
  model_name = "bayes_prior_horse"
)

metrics_p2 <- calc_metrics_bayes(
  fit = fit_laplace,
  test_data = test_bayes,
  model_name = "bayes_prior_l1"
)

metrics_p3 <- calc_metrics_bayes(
  fit = fit_normal,
  test_data = test_bayes,
  model_name = "bayes_prior_normal"
)

```

```
summary(fit_laplace)
```

Model Info:

```

function:      stan_glm
family:        binomial [logit]
formula:       PD30 ~ .
algorithm:      sampling
sample:        2000 (posterior sample size)
priors:         see help('prior_summary')
observations:  22500
predictors:    24

```

Estimates:

	mean	sd	10%	50%	90%
(Intercept)	-1.5	0.0	-1.5	-1.5	-1.4
limite_credito	-0.1	0.0	-0.1	-0.1	-0.1
sexo	0.0	0.0	-0.1	0.0	0.0
escolaridade	-0.1	0.0	-0.1	-0.1	-0.1
estado_civil	-0.1	0.0	-0.1	-0.1	-0.1
idade	0.1	0.0	0.0	0.1	0.1
atraso_pagto_0	0.6	0.0	0.6	0.6	0.7
atraso_pagto_2	0.1	0.0	0.1	0.1	0.2
atraso_pagto_3	0.1	0.0	0.1	0.1	0.1
atraso_pagto_4	0.0	0.0	0.0	0.0	0.1
atraso_pagto_5	0.1	0.0	0.0	0.1	0.1
atraso_pagto_6	0.0	0.0	0.0	0.0	0.0
fatura_valor_1	-0.4	0.1	-0.5	-0.4	-0.3
fatura_valor_2	0.2	0.1	0.0	0.2	0.3
fatura_valor_3	0.1	0.1	0.0	0.1	0.2
fatura_valor_4	-0.1	0.1	-0.2	-0.1	0.1
fatura_valor_5	0.1	0.1	0.0	0.1	0.2
fatura_valor_6	0.0	0.1	-0.1	0.0	0.1
pagto_valor_1	-0.2	0.0	-0.3	-0.2	-0.2
pagto_valor_2	-0.2	0.0	-0.2	-0.2	-0.1
pagto_valor_3	-0.1	0.0	-0.1	-0.1	-0.1
pagto_valor_4	0.0	0.0	-0.1	0.0	0.0
pagto_valor_5	0.0	0.0	-0.1	0.0	0.0
pagto_valor_6	0.0	0.0	-0.1	0.0	0.0

Fit Diagnostics:

	mean	sd	10%	50%	90%
mean_PPD	0.2	0.0	0.2	0.2	0.2

The mean_ppd is the sample average posterior predictive distribution of the outcome variable

MCMC diagnostics

	mcse	Rhat	n_eff
(Intercept)	0.0	1.0	2472
limite_credito	0.0	1.0	1575
sexo	0.0	1.0	1862
escolaridade	0.0	1.0	2121
estado_civil	0.0	1.0	1795
idade	0.0	1.0	2036
atraso_pagto_0	0.0	1.0	1922
atraso_pagto_2	0.0	1.0	2159
atraso_pagto_3	0.0	1.0	1949
atraso_pagto_4	0.0	1.0	1748
atraso_pagto_5	0.0	1.0	1997
atraso_pagto_6	0.0	1.0	1698
fatura_valor_1	0.0	1.0	1952
fatura_valor_2	0.0	1.0	1871
fatura_valor_3	0.0	1.0	1602
fatura_valor_4	0.0	1.0	1256
fatura_valor_5	0.0	1.0	1348
fatura_valor_6	0.0	1.0	1586
pagto_valor_1	0.0	1.0	2064
pagto_valor_2	0.0	1.0	2033
pagto_valor_3	0.0	1.0	1817
pagto_valor_4	0.0	1.0	1691
pagto_valor_5	0.0	1.0	1658
pagto_valor_6	0.0	1.0	2181
mean_PPD	0.0	1.0	2576
log-posterior	0.5	1.0	249

For each parameter, mcse is Monte Carlo standard error, n_eff is a crude measure of effective

```
summary(fit_horseshoe)
```

Model Info:

```
function:      stan_glm
family:        binomial [logit]
formula:       PD30 ~ .
algorithm:     sampling
sample:        2000 (posterior sample size)
priors:        see help('prior_summary')
observations:  22500
predictors:    24
```

Estimates:

	mean	sd	10%	50%	90%
(Intercept)	-1.5	0.0	-1.5	-1.5	-1.4
limite_credito	-0.1	0.0	-0.1	-0.1	-0.1
sexo	0.0	0.0	-0.1	0.0	0.0
escolaridade	-0.1	0.0	-0.1	-0.1	0.0
estado_civil	-0.1	0.0	-0.1	-0.1	-0.1
idade	0.1	0.0	0.0	0.1	0.1
atraso_pagto_0	0.7	0.0	0.6	0.7	0.7
atraso_pagto_2	0.1	0.0	0.1	0.1	0.2
atraso_pagto_3	0.1	0.0	0.1	0.1	0.1
atraso_pagto_4	0.0	0.0	0.0	0.0	0.0
atraso_pagto_5	0.1	0.0	0.0	0.1	0.1
atraso_pagto_6	0.0	0.0	0.0	0.0	0.0
fatura_valor_1	-0.3	0.1	-0.4	-0.3	-0.2
fatura_valor_2	0.1	0.1	0.0	0.1	0.2
fatura_valor_3	0.1	0.1	0.0	0.0	0.2
fatura_valor_4	0.0	0.1	0.0	0.0	0.1
fatura_valor_5	0.0	0.1	0.0	0.0	0.1
fatura_valor_6	0.0	0.0	0.0	0.0	0.1
pagto_valor_1	-0.2	0.0	-0.3	-0.2	-0.1
pagto_valor_2	-0.2	0.0	-0.2	-0.2	-0.1
pagto_valor_3	-0.1	0.0	-0.1	-0.1	0.0
pagto_valor_4	0.0	0.0	-0.1	0.0	0.0
pagto_valor_5	0.0	0.0	-0.1	0.0	0.0
pagto_valor_6	0.0	0.0	-0.1	0.0	0.0

Fit Diagnostics:

	mean	sd	10%	50%	90%
mean_PPD	0.2	0.0	0.2	0.2	0.2

The mean_ppd is the sample average posterior predictive distribution of the outcome variable

MCMC diagnostics

	mcse	Rhat	n_eff
(Intercept)	0.0	1.0	3305
limite_credito	0.0	1.0	2014
sexo	0.0	1.0	1510
escolaridade	0.0	1.0	1703
estado_civil	0.0	1.0	2032
idade	0.0	1.0	1739
atraso_pagto_0	0.0	1.0	1943

```

atraso_pagto_2 0.0 1.0 1991
atraso_pagto_3 0.0 1.0 1858
atraso_pagto_4 0.0 1.0 1777
atraso_pagto_5 0.0 1.0 1540
atraso_pagto_6 0.0 1.0 2272
fatura_valor_1 0.0 1.0 1613
fatura_valor_2 0.0 1.0 1669
fatura_valor_3 0.0 1.0 1548
fatura_valor_4 0.0 1.0 1904
fatura_valor_5 0.0 1.0 1514
fatura_valor_6 0.0 1.0 2017
pagto_valor_1 0.0 1.0 1966
pagto_valor_2 0.0 1.0 1563
pagto_valor_3 0.0 1.0 1629
pagto_valor_4 0.0 1.0 1824
pagto_valor_5 0.0 1.0 1236
pagto_valor_6 0.0 1.0 1341
mean_PPD      0.0 1.0 2420
log-posterior 0.3 1.0 523

```

For each parameter, mcse is Monte Carlo standard error, n_eff is a crude measure of effective

```

metrics_table_all <- bind_rows(
  metrics_table_final,
  metrics_p1,
  metrics_p2,
  metrics_p3,
)
metrics_table_all

```

A tibble: 7 × 15

	ac- cu-	bal_ac- cu-	roc_auc	log_loss	rier	sens	spec	sion	npv	f1	f2	kap	mcc	gmean
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
all_features_logit	0.8076006	0.0052357	1.365087	1.695947	1.039681	0.624230	0.481810	0.268367	0.386850	0.233908	0.273737	0.162498	0.184353	0.1643536
no_confusion	0.8074067	0.0051098	1.120449	1.208754	0.739626	0.798424	0.230481	0.068250	0.286762	0.293370	0.373432	0.224470	0.148431	0.10843108
l1_logit	0.8074067	0.0042867	1.323547	1.593147	1.079686	0.697399	0.381774	0.285025	0.386833	0.341872	0.717073	0.238290	0.20657	0.20657
l2_logit	0.8038667	0.0093559	1.136747	1.807147	0.757970	0.721639	0.581348	0.667735	0.588510	0.634504	0.424738	0.303062	0.258323	0.2583233

	ac- cu-	bal_ac- cu-					pre- ci-							
wflow	roc	racy	roc	log_loss	brier	sens	spec	sion	npv	f1	f2	kap	mcc	gmean
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
bayes_p	0.8078666047592129305517384	0.8078666047592129305517384	0.8078666047592129305517384	0.8078666047592129305517384	0.8078666047592129305517384	0.9690122405068179096879918870079344862731063255534827562	0.9690122405068179096879918870079344862731063255534827562	0.9690122405068179096879918870079344862731063255534827562	0.9690122405068179096879918870079344862731063255534827562	0.9690122405068179096879918870079344862731063255534827562	0.9690122405068179096879918870079344862731063255534827562	0.9690122405068179096879918870079344862731063255534827562	0.9690122405068179096879918870079344862731063255534827562	
bayes_p	0.80746667047082137742585211	0.80746667047082137742585211	0.80746667047082137742585211	0.80746667047082137742585211	0.80746667047082137742585211	0.96830724110981799326837608867989339802726053241094831900	0.96830724110981799326837608867989339802726053241094831900	0.96830724110981799326837608867989339802726053241094831900	0.96830724110981799326837608867989339802726053241094831900	0.96830724110981799326837608867989339802726053241094831900	0.96830724110981799326837608867989339802726053241094831900	0.96830724110981799326837608867989339802726053241094831900	0.96830724110981799326837608867989339802726053241094831900	
bayes_p	0.8078666054065138345589486	0.8078666054065138345589486	0.8078666054065138345589486	0.8078666054065138345589486	0.8078666054065138345589486	0.9684982423048181956860068870249341062743803260254844392	0.9684982423048181956860068870249341062743803260254844392	0.9684982423048181956860068870249341062743803260254844392	0.9684982423048181956860068870249341062743803260254844392	0.9684982423048181956860068870249341062743803260254844392	0.9684982423048181956860068870249341062743803260254844392	0.9684982423048181956860068870249341062743803260254844392	0.9684982423048181956860068870249341062743803260254844392	
mal														